I/O Control API

Development Documentation

V1.1.0

Update Records

Date	Description
2022-10-31	First Edition Release
2023-02-22	Update API
2023-04-23	Update API and demo
2024-06-20	Update API and documentation

Contents

1 Introduction

Package: (com.common.apiutil.pos)

Mainly for LED, Relay, Wiegand, RS232/485, Door Magnetic, Physical key to make a brief description

1.1 LED

1.1.1 LED Control

```
/**
 * @Function Led Control
 *
 * @Parameter
 * 1.ledType: LED Type: please refer the quick guidance document.
 * 2.ledColor: LED Color: please refer the quick guidance document.
 * 3.brightness: brightness: [0 - 255], If it cannot be adjusted, and 0: Close;
larger than 0: Open;
 *
 * @Return ResultCode: please refer the quick guidance document.
 *
 * */

public static int setColorLed(int ledType, int ledColor, int brightness);
```

1.2 Relay

1.2.1 Relay power on/off

```
/**
 * @Function Relay Control
 *
 * @Parameter
 * 1.type: Relay Type: please refer the quick guidance document.
 * 2.status: Close:0, Open:1
 *
 * @Return ResultCode: please refer the quick guidance document.
 *
 * */

public static int setRelayPower(int type, int status)
```

1.3 GPIO Control

1.3.1 Set the GPIO I/O

```
/**
 * @Function Set the GPIO Input/Output
 *
 * @Parameter
 * 1.type: GPIO Type: please refer the quick guidance document.
 * 2.direction: GPIO Direction: please refer the quick guidance document.
 *
 * @Return ResultCode: please refer the quick guidance document.
 *
 * */

public static int setGPIOControl(int type, int direction)
```

1.3.2 Set the GPIO Level

```
/**
 * @Function Set the GPIO Level
 *
 * @Parameter
 * 1.type: GPIO Type: please refer the quick guidance document.
 * 2.level: GPIO Level: please refer the quick guidance document.
 *
 * @Return ResultCode: please refer the quick guidance document.
 *
 * */

public static int setGPIOLevel(int type, int level)
```

1.3.3 Get the GPIO Status

```
/**
 * @Function Get the GPIO Status, Include input/output status and level status
 *
 * @Parameter
 * 1.type: GPIO Type: please refer the quick guidance document.
 *
 * @Return
 * GPIO Direction: please refer the quick guidance document.
 * GPIO Level: please refer the quick guidance document.
 * */

public synchronized int[] getGPIOStatus(int type)
```

1.4 Door/Magnetic Sensor/Physical Key/Wiegand

1.4.1 Get the status of the sensor

```
/**
 * @Function Gets the status of the sensor
 *
 * @Parameter
 * 1.type:
 *      0 -> Get the IR sensor status, 1: someone;0:none
 *      1 -> Get the removal sensor status, 1: dismantle;0:not dismantle
 *      2 -> Get the physical button door opening signal, 1: High level;0: Low
level
 *      3 -> Acquire a magnetic susceptibility circuit signal, 1: High level;0:
Low level
 *      4 -> Acquire a light sensor signal, 1:light;0: No light
 *      5 -> Obtain the bottom shell tamper signal, 1: dismantle;0: not dismantle
 *      6 -> Get the microwave status, 1:someone; 0:none
 *      7 -> Get the door status, 1:Open; 0:Close
 *      8 -> Get the door lock status, 1:Open;0:Close
 *      9 -> Obtain the magnetic induction circuit 1 state, 1:Open;0:Close
 *      10 -> Obtain the magnetic induction circuit 2 state , 1:Open;0:Close
 *
 * @Return 1; 0; -1, Failure
 *
 * */

public synchronized void getPriximitySensorStatus(int type)
```

1.4.2 Wiegand Send

```
/**
 * @Function Transmit Wiegand signals
 *
 * @Parameter
 * 1.type: Wiegand Type: please refer the quick guidance document.
 * 2.data: For hexadecimal data, the 26-bit Wiegand valid data bit is 24 bits, the
34-bit Wiegand valid data bit is 32 bits, and the custom Wiegand is passthrough
 *
 * @Return ResultCode: please refer the quick guidance document.
 *
 * */

public synchronized int wiegandSend(int type, byte[] data)
```

1.4.3 Wiegand I/O Switch

```
/**
 * @Function Wiegand I/O Switch
 *
 * @Parameter
 * 1.direction: Signal Type: please refer the quick guidance document.
```

```
 *
 * @Return ResultCode: please refer the quick guidance document.
 *
 * */

public synchronized int setWiegandDirection(int direction)
```

1.4.4 Input Signal Switch

```
/**
 * @Function Toggle the input switch
 *
 * @Parameter
 * 1.input1: Input Type: please refer the quick guidance document.
 * 2.input2: Input Type: please refer the quick guidance document.
 * 3.sw: Input Status: please refer the quick guidance document.
 *
 * @Return ResultCode: please refer the quick guidance document.
 *
 * */

public synchronized int switchInput(String input1, String input2, int sw)
```

1.4.5 Register Wiegand/Key/Door input broadcast

```
/**
 * @Function Register Wiegand/Key/Door input broadcast
 *
 * @Parameter
 * 1.context
 *
 * @Return void
 *
 * */

public synchronized void registerInputBroadcast(Context context)
```

1.4.6 Unregister Wiegand/Key/Door input broadcast

```
/**
 * @Function Unregister Wiegand/Key/Door input broadcast
 *
 * @Return void
 *
 * */
```

```
public synchronized void unRegisterInputBroadcast()
```

1.4.7 Set up Wiegand/Key/Door listening

```
/**
 * @Function Set up Wiegan/Key/Door listening
 *
 * @Parameter
 * 1.inputListener: IInputListener
 *
 * @Return void
 *
 * */

public synchronized void setInputListener(IInputListener inputListener)
```

1.4.8 Receive the listening class

```
public interface IInputListener {
    /**
     * @Function Wiegand input
     *
     * @Parameter inputData
     *
     * */
    void wiegandInput(byte[] inputData);

    /**
     * @Function Magnetic door/Physical key/Magnetic Input
     *
     * @Parameter
     * 1.sw:
     *      1: Magnetic door
     *      2: Physical key
     *      3: Magnetic Input1
     *      4: Magnetic Input2
     * 2.status: 0:low level; 1:high level
     *
     * */
    void input(int sw, int status);
}
```

1.5 RS232Reader/RS485Reader/RSTTLReader

(com.common.apiutil.pos) RS232/RS485/RSTTL

1.5.1 Get the RS232 control object

```
/**
 * @Function Get the RS232 control object
 *
 * @Parameter context
 *
 * */

public RS232Reader(Context context)
```

1.5.2 Get the RS485 control object

```
/**
 * @Function Get the RS485 control object
 *
 * @Parameter context
 *
 * */

public RS485Reader(Context context)
```

1.5.3 Get the RSTTL control object

```
/**
 * @Function Get the RSTTL control object
 *
 * @Parameter context
 *
 * */

public RSTTLReader(Context context)
```

1.5.4 Open

```
/**
 * @Function Open RS232/RS485/RSTTL
 *
 * @Parameter
 * 1.type: RS232/RS485/RSTTL Type: please refer the quick guidance document.
 * 2.baud: baud rate
 *
 * @Return ResultCode: please refer the quick guidance document.
```

```
 *
 * */

public synchronized int rsOpen(int type, int baud)
```

1.5.5 Destroy

```
/**
 * @Function Destroy RS232/RS485/RSTTL
 *
 * @Return ResultCode: please refer the quick guidance document.
 *
 * */

public synchronized int rsDestroy()
```

1.5.6 Send data

```
/**
 * @Function Send the RS232/RS485/RSTTL data
 *
 * @Parameter data
 *
 * @Return ResultCode: please refer the quick guidance document.
 *
 * */

public synchronized int rsSend(byte[] data)
```

1.5.7 Set up receiving listening

```
/**
 * @Function Set up receiving monitoring
 *
 * @Parameter
 * 1.listenser: IRSReaderListener
 *
 * @Return void
 *
 * */

public synchronized void setRSReaderListener(IRSReaderListener listener)
```

1.5.8 RS232/RS485/RSTTL IRSReaderListener

```
/**
 * @Function RS232/RS485/RSTTL Receive the listening class
 *
 * */

public synchronized interface IRSReaderListener{
    /**
     * @Parameter data
     *
     * */
    void onRecvData(byte[] data);
}
```

1.5.9 Set the RS485 transceiver mode

```
/**
 * @Function Set the RS485 transceiver mode
 *
 * @Parameter
 * 1.mode: RSMode: please refer the quick guidance document.
 *
 * @Return ResultCode: please refer the quick guidance document.
 *
 * */

public int setMode(int mode)
```

1.5.10 Get the serial port number path

```
/**
 * @Function Get the serial port number path
 *
 * @Parameter
 * 1.type: RS232/RS485/RSTTL
 *
 * @Return serial port number path
 *
 * */

public synchronized String getSerialPath(int type)
```